Fachhochschule Nordwestschweiz
Hochschule für Technik

Optimizers

Deep Learning

HS 2025

Martin Melchior

Overview on Optimisers

just some of the most prominent optimisers

Stochastic Gradient Descent

Vanilla SGD

```
torch.optim.SGD(params, lr)
```

Momentum, Nesterov

Designed to surpass flat regions or saddle points

```
torch.optim.SGD(params, lr, momentum, nesterov)
```

RMS Prop

Adaptively adjusts the learning rate to incorporate variations in the steepness

```
torch.optim.RMSProp(params, lr, alpha)
```

Adam

~ state of the art

Combination of Momentum / Nesterov and RMSprop.

```
torch.optim.Adam(params, lr, alpha, betas)
```

Optimizers in PyTorch

choose optimiser
set its parameters (e.g. lr)
let it know the model parameters

```
optimizer = torch.optim.SGD(model.parameters(), lr=lr)

for epoch in range(nepochs):
    model.train()
    size = len(trainloader.sampler)
    size_test = len(testloader.sampler)
    cost, acc = 0.0, 0.0
    for batch, (X, Y) in enumerate(trainloader):
        pred = model(X)
        loss = cost_fn(pred, Y)
        cost += loss.item()
        acc += (pred.argmax(dim=1) == Y).type(torch.float).sum().item()

        # gradient, parameter update
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

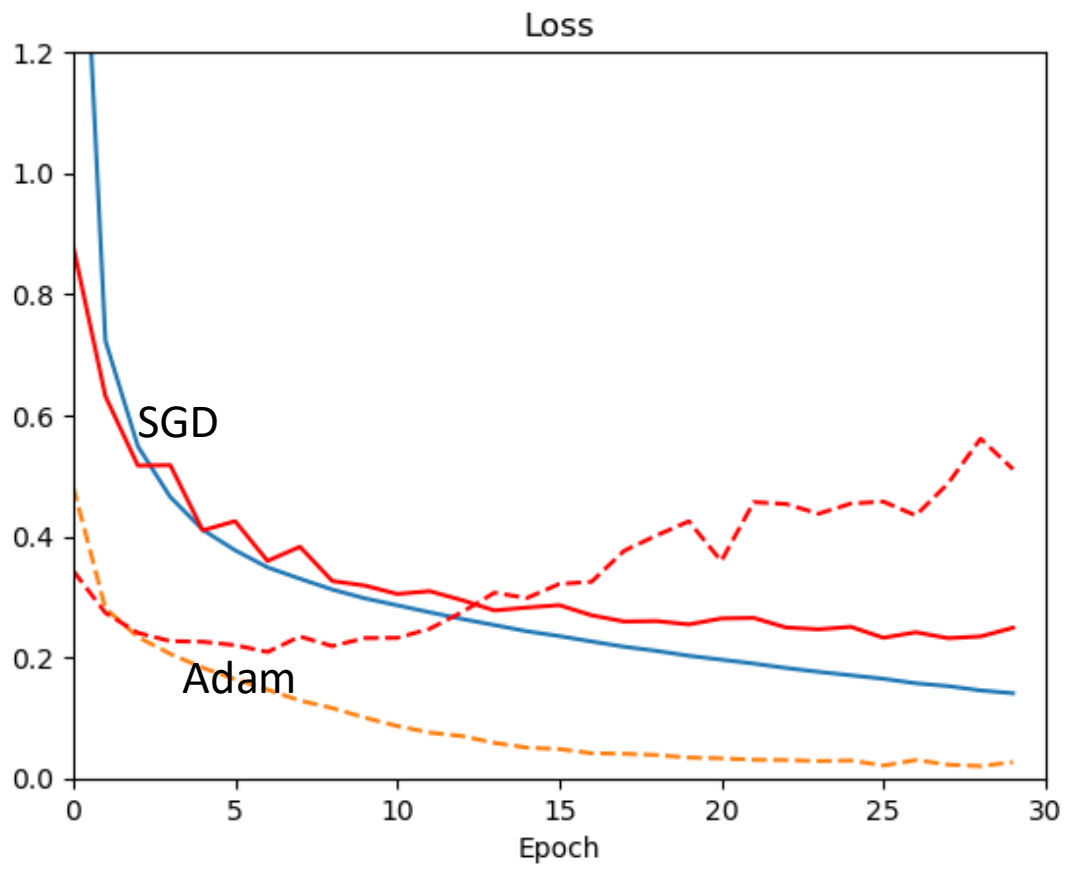
reset cached gradients from previous batch

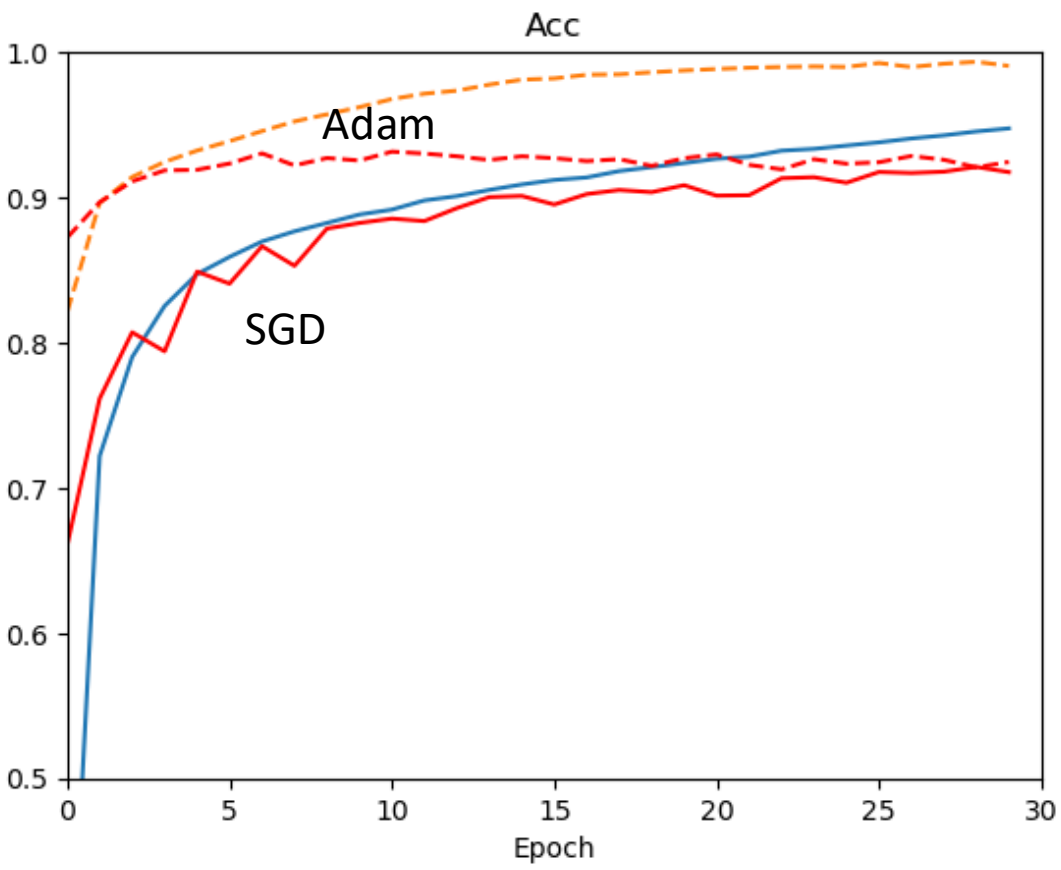
other steps remain the same

compute the gradients with backprop (grad attached to tensors with model params)

update the parameters after backprop

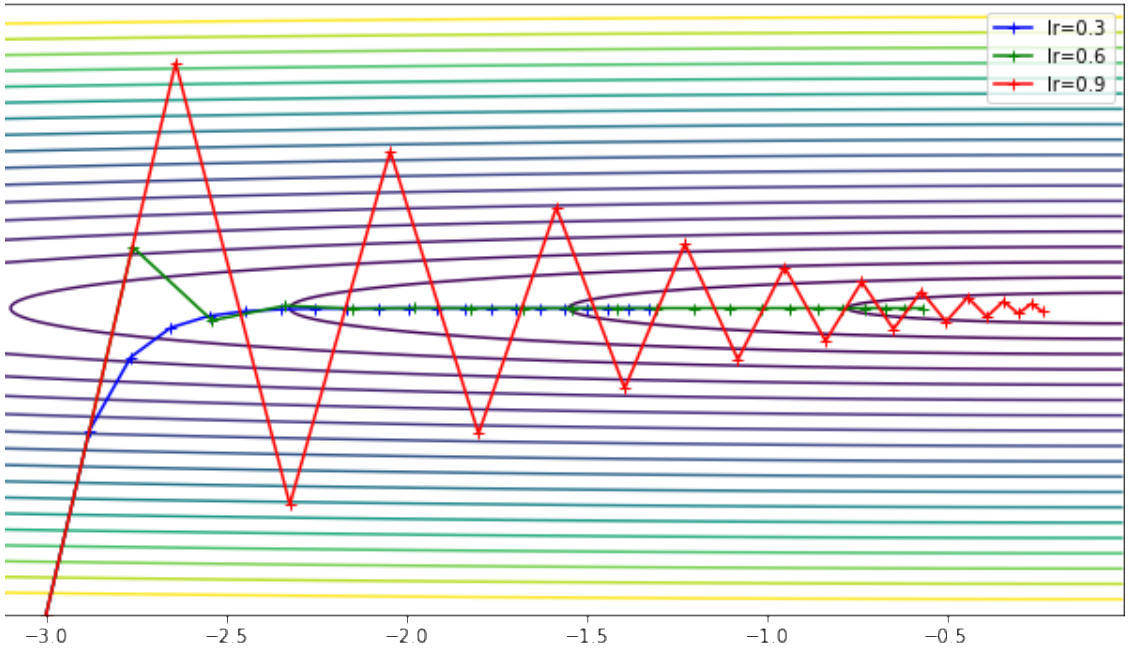
Adam in Action





Weakness of Plain Gradient Descent

Plain gradient descent makes a strong assumption about the magnitude of the “local curvature” to fix the step size, and about its isotropy so that the same step size makes sense in all directions.



With fixed learning rate:

Flat regions
-> small gradients
-> small steps

Steep regions
-> large gradients
-> large steps

Toy example:

$$f(x) = \frac{x^2}{15} + y^2$$

Gradient Descent in Deep Learning Models

Some optimisation methods use higher-order derivatives (2nd order Hessian). These provide a more informative local approximation in view of defining the update steps.

In Deep Learning Problems:

- Higher-order methods not feasible due to the high-dimensional parameter space and the large(r) computational complexity of the methods.
- Use gradients in a smarter way - statistics over its past values to make "smarter steps".

Similar parameter update rule: $\theta^{(t+1)} = \theta^{(t)} - G)$ G :step in descent direction)

Momentum

- Modified scheme for computing the descent direction (used in the parameter update step)

- Intuition

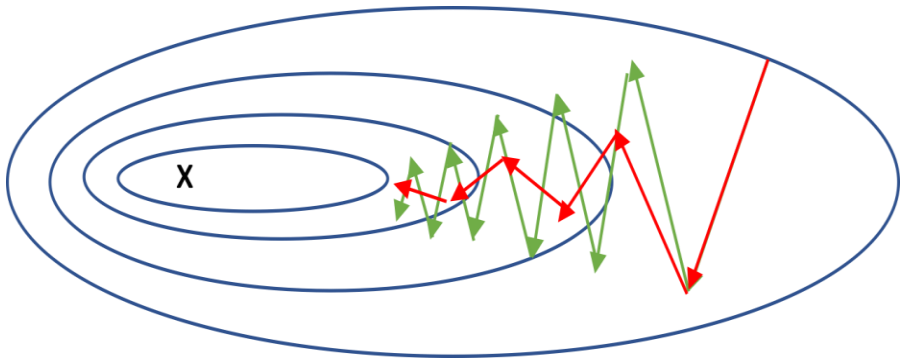
Imagine a ball rolling down a gentle slope on a smooth surface. It starts out slowly but quickly picks up momentum until it eventually reaches a terminal velocity (if there is some friction or air resistance). …
In contrast, regular Gradient Descent will simply take small regular steps down the slope, so it will take much more time to reach the bottom.

(from A. Géron, 'Hands-On Machine Learning with Scikit-Learn and TensorFlow')

- Algorithm: Compute an exponentially decaying moving average of past gradients and move in the direction of the moving average.

Momentum

- Typical Learning Trajectories in Parameter Space with/without momentum



Plain gradient descent : green
Gradient descent with momentum: red
With momentum, minimum at ‘x’ is more directly approached. Due to the moving average, the vertical component of the gradient is more and more eliminated.

- Implementation affects just the parameter update rule (descent direction), i.e. the optimiser, but not the model’s forward prop and backprop mechanics.

Momentum, Nesterov

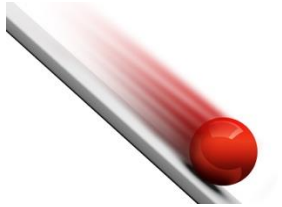
$$
\begin{aligned}
m^{(t+1)} &= \beta_1 m^{(t)} + \nabla C(\theta^{(t)}) \\
\theta^{(t+1)} &= \theta^{(t)} - \alpha m^{(t+1)}
\end{aligned}
$$

'momentum' hyper-parameter which controls the decay and the friction: small values of β_1 means large friction and fast decay.
Good default: $\beta_1 = 0.9$

With **Nesterov**, this term is modified to

$$\alpha \nabla C(\theta^{(t)} + \beta_1 m^{(t)})$$

which is considered to be more efficient.



For constant gradient v we obtain: $m \approx (1-\beta_1)^{-1} v$ (for large t).
Hence, the gradient is magnified by factor $(1-\beta_1)^{-1}$.

▶ Rather choose smaller learning rate.

Momentum, Nesterov: Implementation in PyTorch

Same optimizer as plain gradient descent

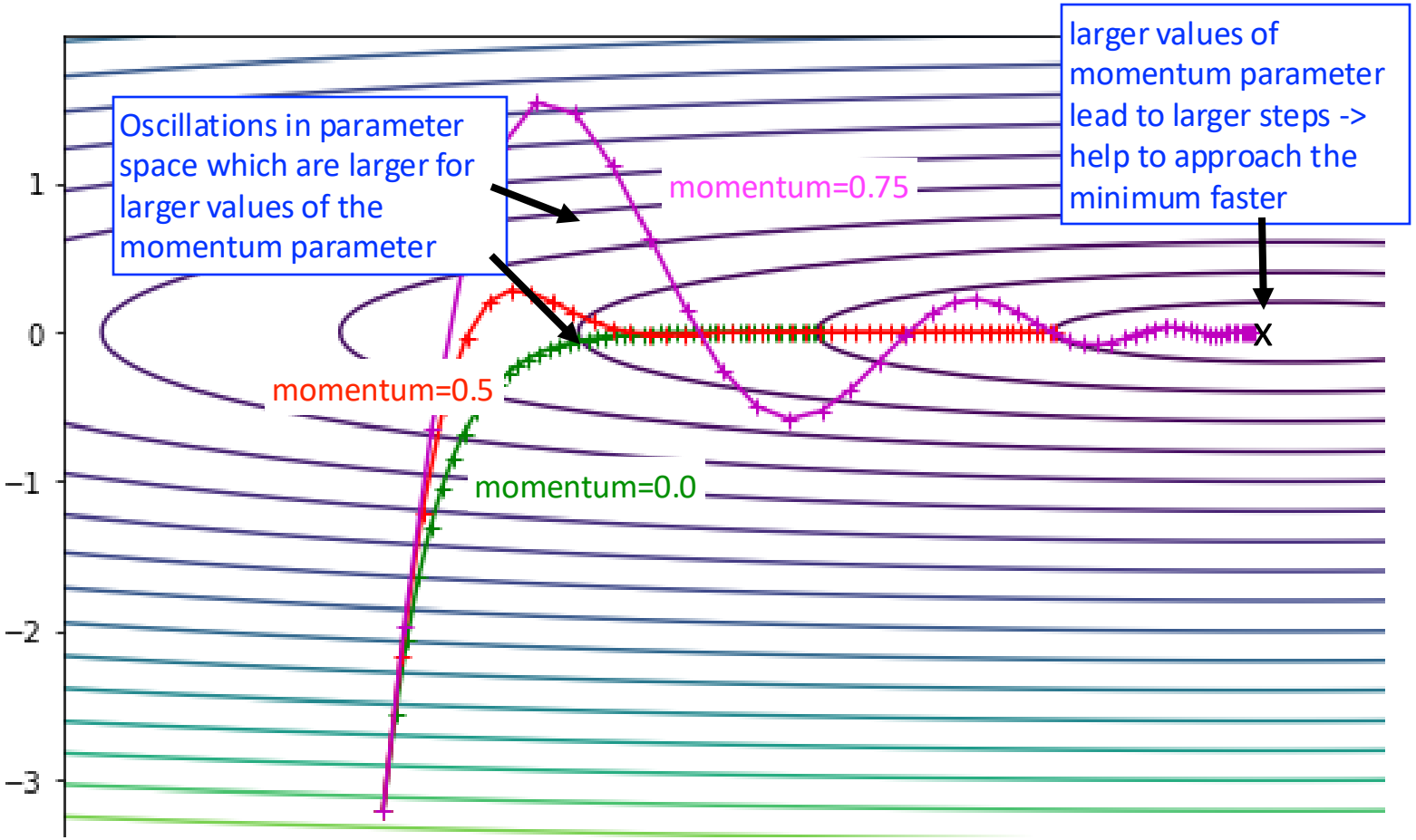
```
torch.optim.SGD(params=model.parameters(),
                lr=lr,
                momentum=0.9,
                nesterov=True)
```

```
help(torch.optim.SGD)
```

```
Help on class SGD in module torch.optim.sgd:

class SGD(torch.optim.optimizer.Optimizer)
 |  SGD(params, lr=<required parameter>, momentum=0, dampening=0, weight_decay=0, nesterov=False, *, maximize=False,
foreach: Optional[bool] = None, differentiable=False)
 |
 |  Implements stochastic gradient descent (optionally with momentum).
 |
```

Toy Example



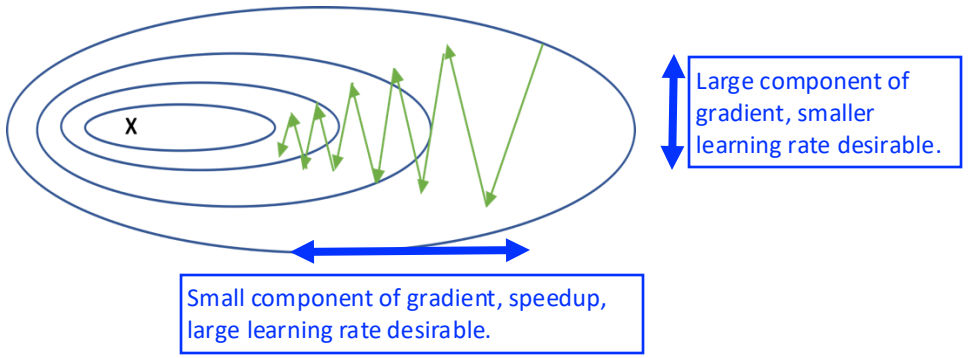
learning rate: 0.1
#epochs: **50**

$$f(x) = \frac{x^2}{15} + y^2$$

RMS Prop

Idea: Adjust the learning rate (adaptively)

- Increase learning rate in directions with small component of gradient
- Decrease learning rate in directions with large component of gradient



Homogenise learning speed in the different directions.

Divide each component of the gradient by its typical length scale:

- Measure the square of the scale from an exponentially decaying moving average of the squared components of the gradient.
- Learning rate is expected to be less dependent on specific model.

First proposed in an online course by G. Hinton

RMS Prop

$$
\begin{aligned}
s^{(t+1)} &= \beta_2 s^{(t)} + (1-\beta_2)\nabla C(\theta^{(t)}) \odot \nabla C(\theta^{(t)}) \\
\theta^{(t+1)} &= \theta^{(t)} - \frac{\alpha}{\sqrt{s^{(t+1)}+\epsilon}} \odot \nabla C(\theta^{(t)})
\end{aligned}
$$

'RMSprop' hyper-parameter which controls the memory on length of the gradient components seen along the optimisation path:

small values of β_2 means fast decay

good default: $\beta_1 = 0.99$

element-wise operations

RMS Prop: Implementation in PyTorch

```
torch.optim.RMSprop(params=model.parameters(),
                    lr = lr,
                    alpha=0.99,
                    centered=False)
```

In pytorch, decay parameter β_2 named α (set to 0.99 by default)

```
help(torch.optim.RMSprop)
```

```
Help on class RMSprop in module torch.optim.rmsprop:

class RMSprop(torch.optim.optimizer.Optimizer)
 |  RMSprop(params, lr=0.01, alpha=0.99, eps=1e-08, weight_decay=0, momentum=0, centered=False,
foreach: Optional[bool] = None, maximize: bool = False, differentiable: bool = False)
 |
 |  Implements
 |
 |  .. math::
 |     \begin{
 |          &\
```

input : α (alpha), γ (lr), θ_0 (params), $f(\theta)$ (objective)
λ (weight decay), μ (momentum), *centered*
initialize : $v_0 \leftarrow 0$ (square average), $\mathbf{b}_0 \leftarrow 0$ (buffer), $g_0^{ave} \leftarrow 0$

for $t = 1$ **to** … **do**
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
if $\lambda \neq 0$
$g_t \leftarrow g_t + \lambda\theta_{t-1}$
$v_t \leftarrow \alpha v_{t-1} + (1-\alpha)g_t^2$
$\tilde{v}_t \leftarrow v_t$
if centered
$g_t^{ave} \leftarrow g_{t-1}^{ave}\alpha + (1-\alpha)g_t$
$\tilde{v}_t \leftarrow \tilde{v}_t - \left(g_t^{ave}\right)^2$
if $\mu > 0$
$\mathbf{b}_t \leftarrow \mu\mathbf{b}_{t-1} + g_t/\left(\sqrt{\tilde{v}_t} + \epsilon\right)$
$\theta_t \leftarrow \theta_{t-1} - \gamma\mathbf{b}_t$
else
$\theta_t \leftarrow \theta_{t-1} - \gamma g_t/\left(\sqrt{\tilde{v}_t} + \epsilon\right)$

return θ_t

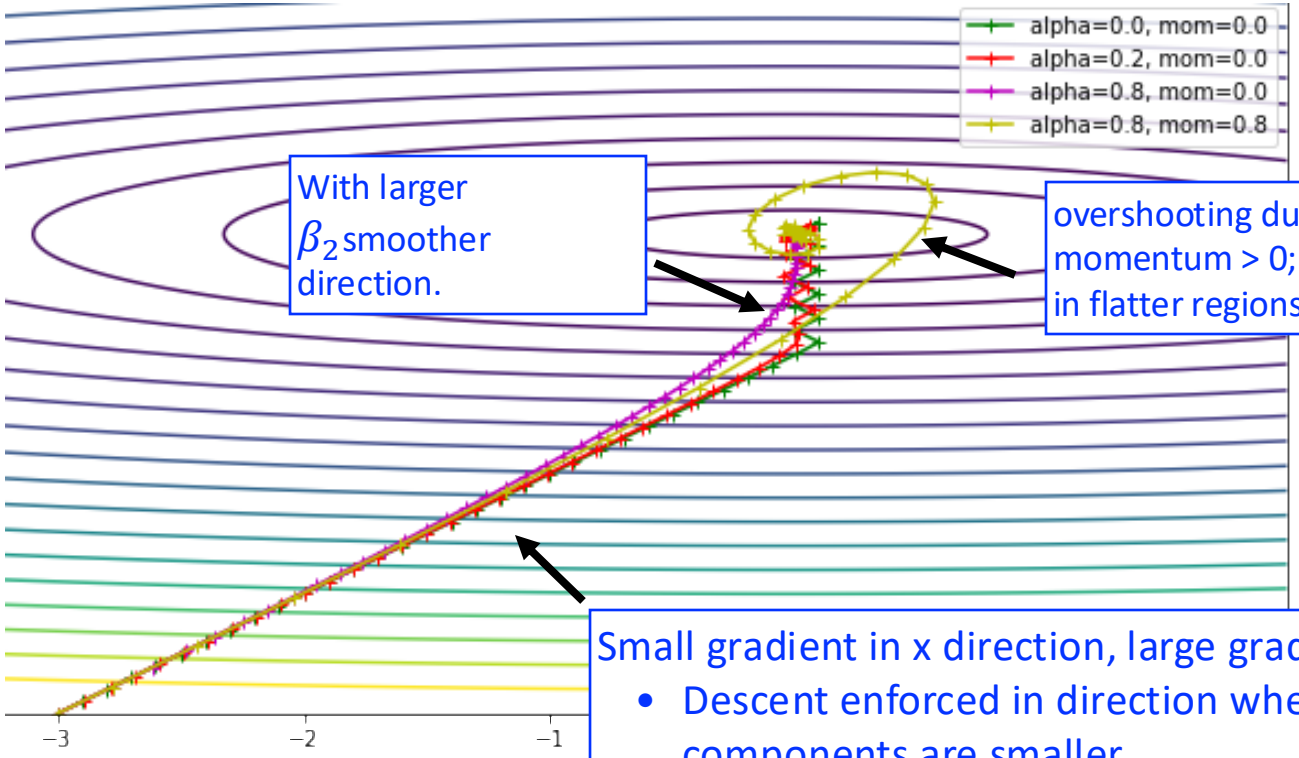
possibly set the momentum parameter

If `centered` is set to `True`, the gradient is normalised by an estimate of its variance (per component).

Operations: Element-wise

Toy Example

$$f(x) = \frac{x^2}{15} + y^2$$



With larger β_2 smoother direction.

overshooting due to momentum > 0; may help in flatter regions

Small gradient in x direction, large gradient in y-direction:

- Descent enforced in direction where gradient components are smaller.
- Effect holds true also for $\beta_2 = 0$!

Adam Optimization

- Combines the ideas of Momentum and RMS Prop
- Considered as the de facto state-of-the-art.
- Algorithm:

$$
\begin{aligned}
m^{(t+1)} &= \beta_1 m^{(t)} + (1-\beta_1)\nabla C(\theta^{(t)}) \\
s^{(t+1)} &= \beta_2 s^{(t)} + (1-\beta_2)\nabla C(\theta^{(t)}) \odot \nabla C(\theta^{(t)}) \\
\hat{m}^{(t+1)} &= \frac{m^{(t+1)}}{1-\beta_1^{t+1}} \\
\hat{s}^{(t+1)} &= \frac{s^{(t+1)}}{1-\beta_2^{t+1}} \\
\theta^{(t+1)} &= \theta^{(t)} - \alpha \frac{\hat{m}^{(t+1)}}{\sqrt{\hat{s}^{(t+1)}+\epsilon}}
\end{aligned}
$$

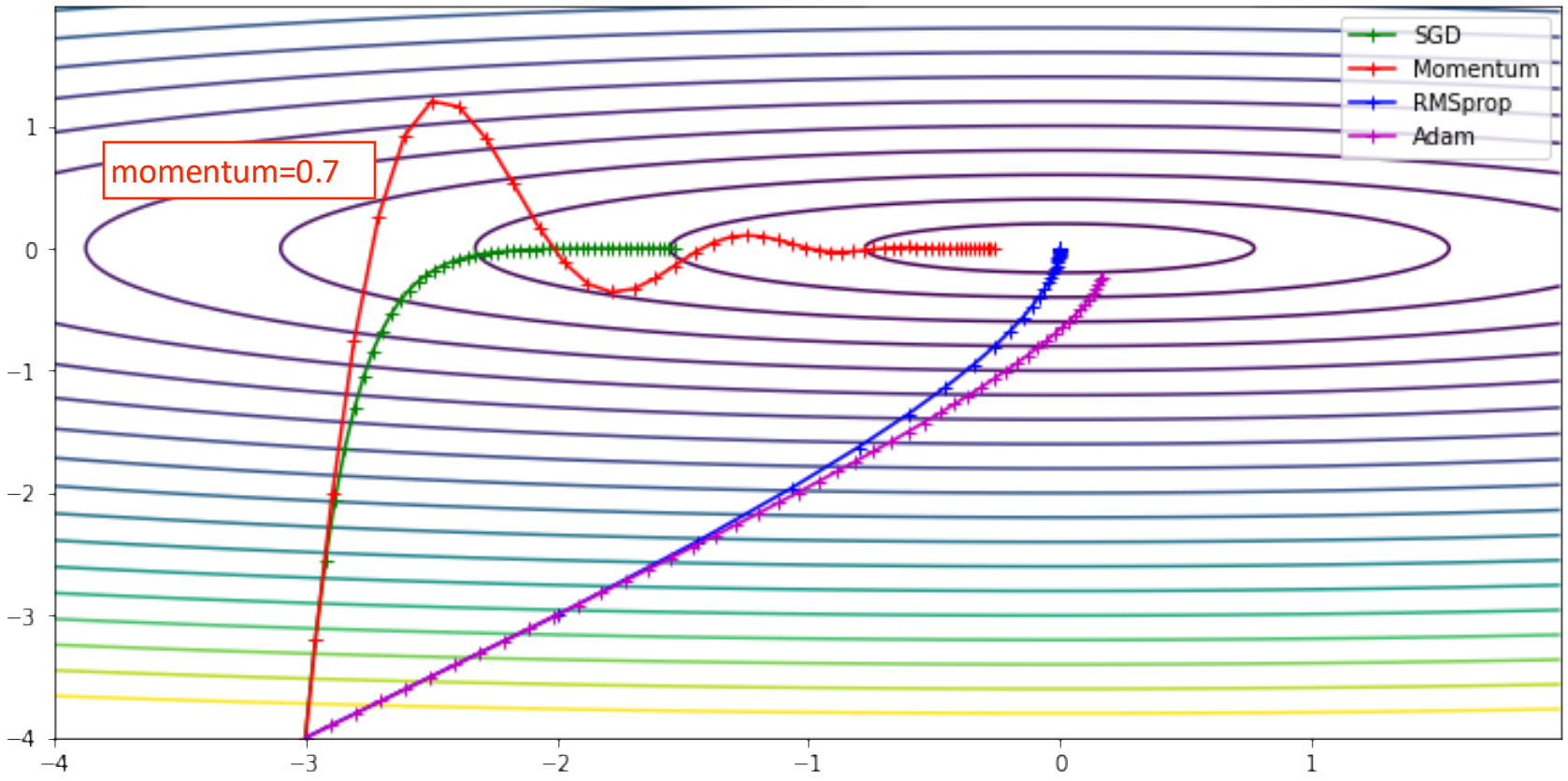
When initialising ***m*** and ***s*** with 0 the normalisation with $1-\beta^n$ gives more weight to the contribution of the current gradient in the start phase.

Element-wise operations applied for the squared gradient, the division and square-root in the update step.

- Hyper-Parameters:

	default
Learning Rate (α)	0.001
Momentum (β_1)	0.9
RMS Decay (β_2)	0.999
Numerical Stabilisation (ε)	1.0E-08

Toy Example

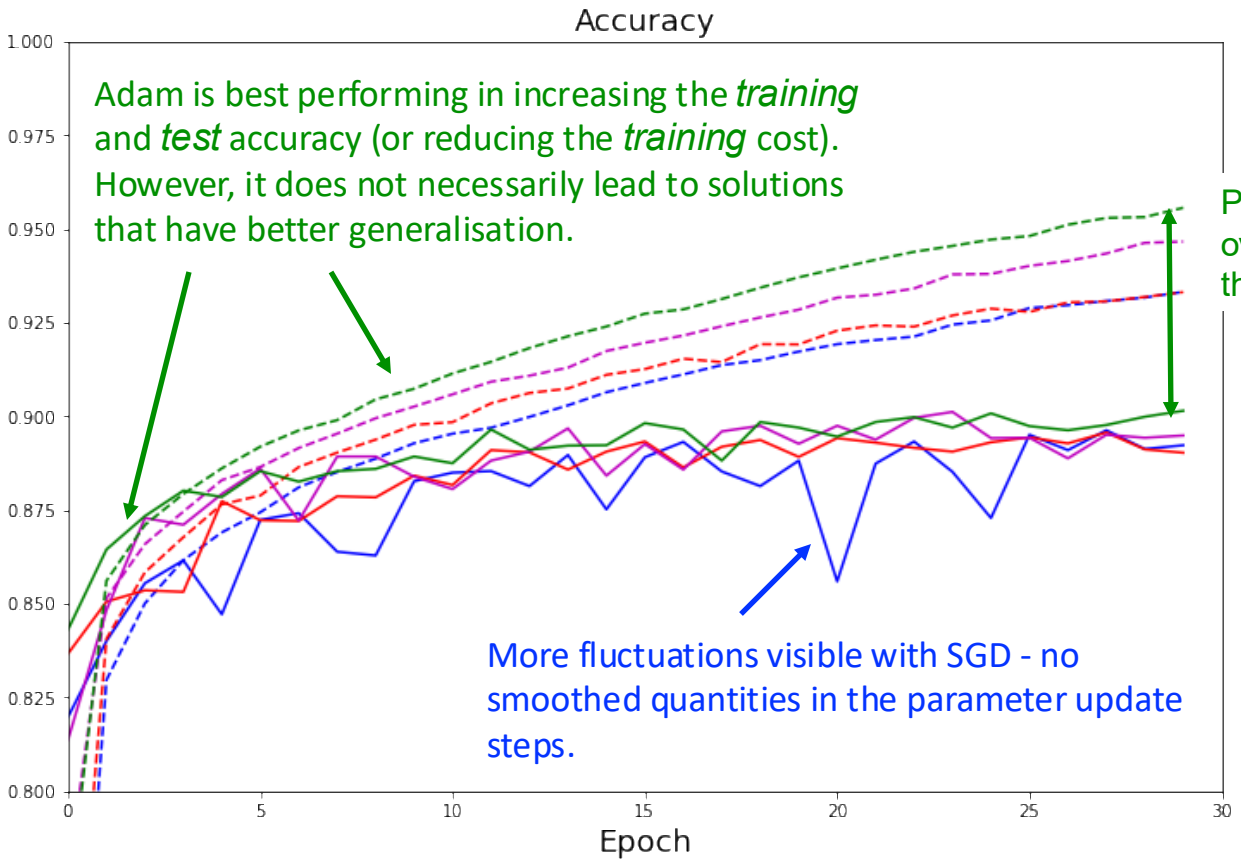


$$f(x) = \frac{x^2}{15} + y^2$$

For all optimisers, function optimised over 50 epochs (iterations) with a learning rate 0.1. Otherwise, default settings were used (i.e. for Momentum, RMSprop and Adam).

Fashion MNIST

MLP with two hidden layers with 250 and 80 units.



Potential danger of overfitting with optimisers that are too performant.

'test' given by the estimate on the 'validation' set split: 50k-10k-10k

SGD:	lr=0.8
Momentum:	lr = 0.1, momentum=0.9
RMSprop:	lr=0.001, alpha=0.95
Adam:	only defaults

Learning Rate Schedule

- Instead of adapting the learning rate in accordance with information collected during the optimisation process (as e.g. in Adam), we may adjust the learning rate according to a rule defined by a function of the epoch number.
- Typically, start with high learning rate, then reduce it.

Piecewise Constant Learning

Epochs	Rate
1-100	0.1
101-300	0.01
301-500	

`torch.optim.lr_scheduler.StepLR`

Exponential scheduling

$$\alpha^{(t+1)} = \gamma\alpha^{(t)} = \cdots = \gamma^{(t+1)}\alpha^{(0)}$$

epoch

`torch.optim.lr_scheduler.ExponentialLR`

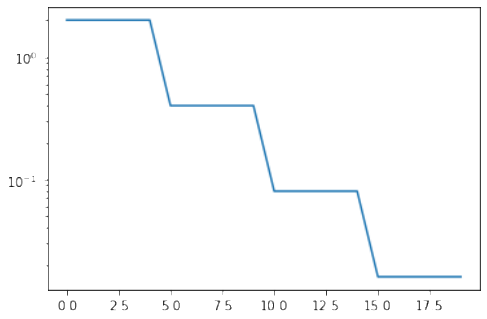
For further options to describe the learning rate decay as a function of the number epochs see: **`torch.optim.lr_scheduler`**

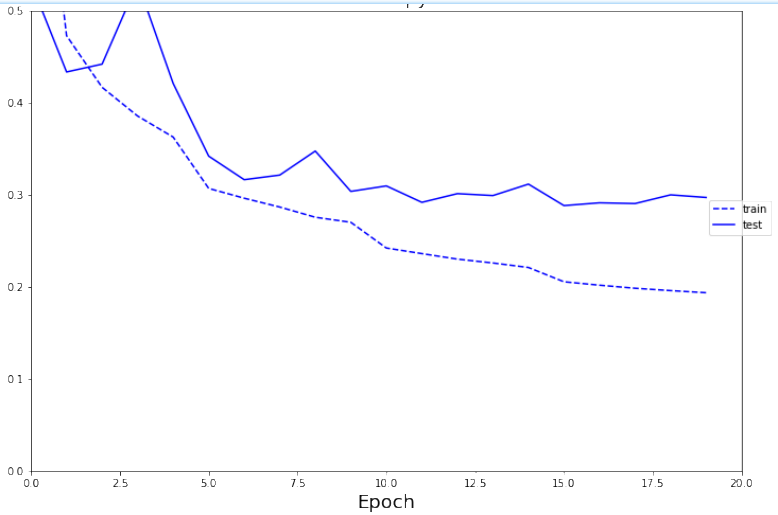
Learning Rate Schedules in PyTorch

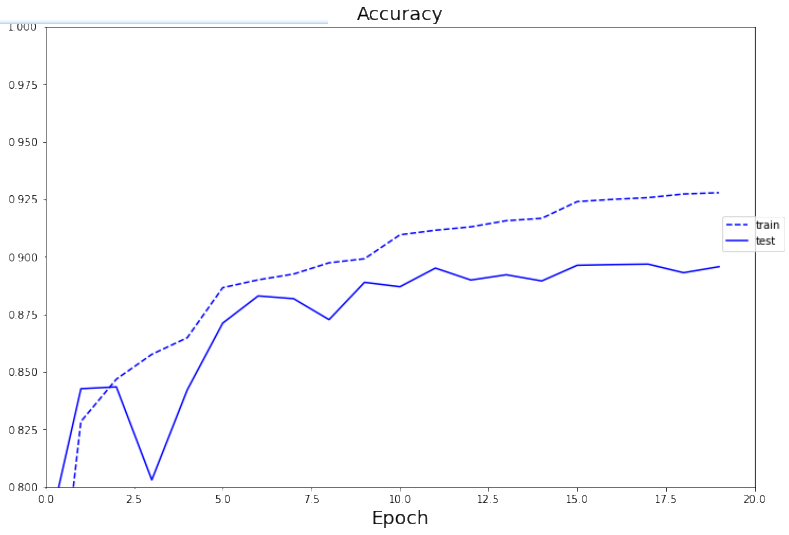
```
model = torch.nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=2.0)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size= 5, gamma=0.2)
nepochs = 20
lrs = []

for i in range(nepochs):
    ## HERE: Model forward / backward pass
    optimizer.step()
    lrs.append(optimizer.param_groups[0]["lr"])
    scheduler.step()

plt.semilogy(range(nepochs),lrs)
```







After every 5 epochs, the learning rate is reduced by a factor 0.2